

HyPlan: An Open-Source Python Library for Planning Airborne Remote Sensing Campaigns

Ryan Pavlick¹✉

¹ National Aeronautics and Space Administration, Washington, DC, USA ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ✉
- [Repository](#) ✉
- [Archive](#) ✉

Editor: [Open Journals](#) ✉

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Airborne remote sensing—using instrumented aircraft to collect data over the Earth’s surface—is a critical tool for Earth science, enabling observations at spatial and temporal scales that bridge ground-based measurements and satellite data. Airborne platforms carry imaging spectrometers, lidars, radars, and other instruments over study areas at altitudes ranging from a few hundred meters to over 20 kilometers, producing data products across Earth science disciplines. Planning these campaigns requires simultaneously reasoning about aircraft flight dynamics, sensor characteristics and ground sampling, environmental conditions, airport logistics, airspace restrictions, satellite overpass coordination, and more. Scientists typically address these constraints using ad hoc spreadsheets, manual calculations, and institutional knowledge, producing workflows that are error-prone, hard to reproduce, and hard to transfer between campaigns.

HyPlan is an open-source Python library that provides a unified, reproducible, and extensible framework for planning airborne remote sensing missions. It encodes the physics of sensor–platform–environment interactions into composable building blocks covering the full mission-planning lifecycle, from study-area definition and flight line generation through swath and GSD calculations to multi-day mission plans with line-ordering optimization. It includes pre-configured models for a range of NASA airborne instruments (e.g., AVIRIS-3, PRISM, LVIS, UAVSAR, HyTES) and multiple research aircraft, along with tools for solar geometry, terrain-aware swath modeling, cloud climatology, airport selection, and satellite overpass prediction.

HyPlan’s core technical contribution is **terrain-aware swath modeling using ray–terrain intersection**, which captures terrain-induced variations in swath width and position that flat-Earth approximations miss; the effect can be substantial in mountainous regions, where swath width may vary by hundreds of meters along a single flight line. A complementary contribution is **wind-aware trajectory modeling subject to aircraft performance constraints**, combining Dubins paths (trochoidal under non-zero wind) with altitude-dependent climb, cruise, and descent models to produce physically realistic flight paths and segment-by-segment timing.

Statement of Need

Airborne science campaigns are major investments—a single ER-2 deployment costs tens of thousands of dollars per flight hour, and campaigns typically span weeks to months—yet the planning process has remained largely manual and fragmented across disconnected tools. Planning these campaigns is a coupled problem spanning sensor performance, solar geometry, environmental conditions, logistics, and satellite coordination. Sensor requirements such as ground sample distance and swath overlap depend on altitude and terrain; solar elevation and glint constrain when observations are viable; clouds and ecosystem state determine data

42 usability; and aircraft endurance and airport availability constrain how flight lines can be
43 scheduled.

44 These constraints are interdependent: altitude affects both sensor performance and coverage,
45 wind alters trajectory and timing, and environmental conditions determine when planned flight
46 lines can be executed. In practice, existing workflows treat these elements separately, making
47 it difficult to evaluate trade-offs or ensure reproducibility.

48 HyPlan addresses this need by treating airborne mission planning as a **coupled physical and**
49 **spatiotemporal problem**, unifying aircraft dynamics, sensor modeling, environmental constraints,
50 and mission-level scheduling within a single reproducible system.

51 State of the Field

52 Several categories of tools address subsets of the airborne mission planning problem, but none
53 in the integrated, programmatic manner that science campaign design requires.

54 **Commercial flight planning software** (ForeFlight, Jeppesen FliteStar) targets pilot navigation,
55 fuel planning, and regulatory compliance, not scientific objectives such as GSD, spectral
56 coverage, or sensor-specific swath overlap. **GIS platforms** (QGIS, ArcGIS) can visualize
57 flight lines and study areas but lack domain-specific calculations for sensor modeling, aircraft
58 performance, and mission timing.

59 **UAS mission planners and agency-internal tools.** Tools such as Mission Planner and
60 QGroundControl ([Dronecode Project, 2024](#)) target small drones at low altitudes and short
61 ranges, and do not model the sensor physics, solar geometry, or logistics constraints relevant
62 to crewed research aircraft. Various NASA centers maintain internal planning tools tailored to
63 specific instruments or aircraft, but these are typically proprietary, undocumented, and tightly
64 coupled to particular missions.

65 **Moving Lines.** The most directly comparable tool is Moving Lines ([LeBlanc, 2018](#)), a Python-
66 based application developed for NASA airborne science campaigns. Moving Lines excels at
67 real-time, interactive flight plan creation and modification during campaign operations, with a
68 graphical user interface combining an interactive map, spreadsheet-based waypoint editing, and
69 overlays of satellite imagery and weather model output. It has been used operationally across
70 numerous NASA campaigns including ORACLES, IMPACTS, and ARCSIX. Moving Lines is
71 designed as a GUI for manual, interactive planning rather than as a programmatic library, and
72 does not provide sensor-specific swath and GSD calculations, terrain-aware analysis, automated
73 flight line generation, or ordering optimization. HyPlan and Moving Lines are complementary:
74 HyPlan addresses the pre-campaign science planning phase—determining how many flight
75 lines are needed, what altitude and speed satisfy sensor requirements, when solar conditions
76 permit data collection, and how to schedule lines across multiple flight days—while Moving
77 Lines supports the tactical, day-of-flight planning and replanning that occurs during campaign
78 execution.

79 Software Design

80 HyPlan is designed as a **programmatic, physics-based flight planning library** for airborne remote
81 sensing campaigns. The architecture reflects a set of deliberate design choices aimed at
82 supporting reproducible, science-driven mission planning while accommodating the complexity
83 of aircraft performance, sensor geometry, and environmental constraints.

84 Programmatic workflows over interactive tools

85 A central design decision is to expose all functionality through a **Python API** rather than
86 a graphical user interface. While interactive tools such as Moving Lines are well-suited for

87 real-time, tactical flight planning, they make it difficult to reproduce, audit, or systematically
88 explore planning decisions.

89 HyPlan instead represents mission design as a sequence of function calls operating on explicit
90 data structures such as FlightLine, FlightBox, and flight plan GeoDataFrames built on
91 NumPy (Harris et al., 2020), pandas (McKinney, 2010), GeoPandas (Jordahl et al., 2020),
92 and Shapely (Gillies & others, 2007). Modules such as flight_plan and flight_optimizer
93 combine these abstractions to produce complete multi-segment sortie descriptions, including
94 takeoff, climb, transit, data collection, and landing.

95 The trade-off is reduced interactivity in exchange for **reproducibility and automation**, enabling
96 version-controlled workflows and systematic exploration of planning parameters.

97 Physics-based modeling over geometric approximation

98 HyPlan explicitly models the physical relationships between aircraft motion, sensor geometry,
99 and the environment. This is reflected across several modules:

- 100 ■ The swath module performs **terrain-aware swath modeling** by tracing rays from the
101 sensor to the terrain surface using digital elevation models accessed via rasterio (Gillies &
102 contributors, 2024) and GDAL (GDAL/OGR contributors, 2024), rather than assuming
103 flat-Earth geometry (Figure 1).
- 104 ■ The flight_plan module incorporates **wind-aware trajectory modeling**, computing crab
105 angles and ground speeds from airspeed and wind vectors (Figure 2).
- 106 ■ The flight planner models **aircraft-constrained motion** by combining a 2D Dubins solver
107 (Dubins, 1957) for horizontal geometry with vertical profiles integrated against horizontal
108 distance from each aircraft's calibrated climb and descent rates, so that top-of-climb
109 and top-of-descent land at physically realistic positions for non-trivial vertical profiles.
110 Under non-zero wind, the air-relative Dubins arcs become **trochoidal ground trajectories**
111 (Sachdev et al., 2023) (Figure 2). A constant-pitch 3D Dubins solver (Vana et al., 2020)
112 is also provided as a reference for short, near-level transits.

AVIRIS-3 @ 25000 foot, 20% overlap — Rincón de la Vieja, Costa Rica

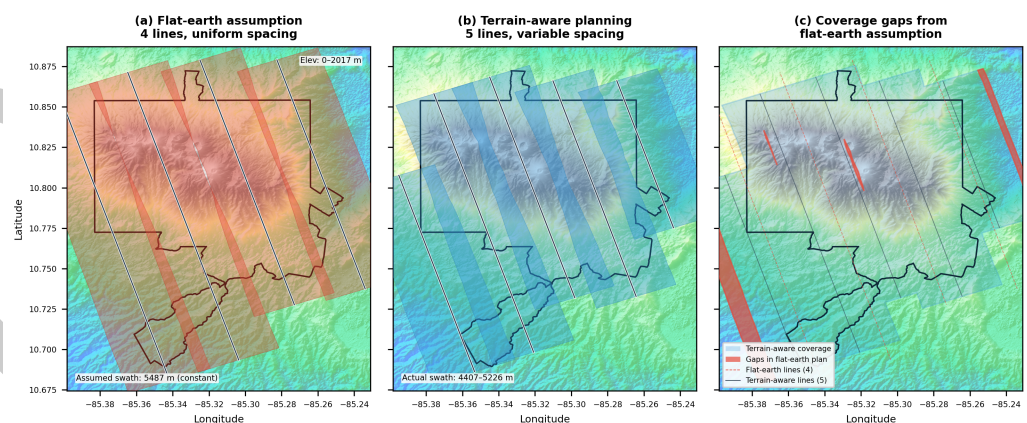


Figure 1: Flat-earth vs terrain-aware flight planning over Rincón de la Vieja National Park, Costa Rica (elevation 234–2,072 m). (a) A flat-earth planner assumes constant swath width and produces uniformly spaced lines. (b) Terrain-aware planning uses ray–terrain intersection to measure actual swath width at each line position, requiring additional lines where terrain narrows the swath. (c) Coverage gaps (red) show areas that would be missed by the flat-earth plan but are covered by the terrain-aware plan.

113 The trade-off is increased complexity compared to planners that assume straight-line motion
114 or constant ground speed. The benefit is **physically realistic trajectory, timing, and coverage**
115 **estimates**—particularly when wind speeds are non-negligible relative to aircraft speed or altitude

changes are required—ensuring that planned trajectories are both kinematically feasible and scientifically valid.

King Air B200, 231 knot cruise, 25.0° bank — MRLB → Rincón de la Vieja → MRLB

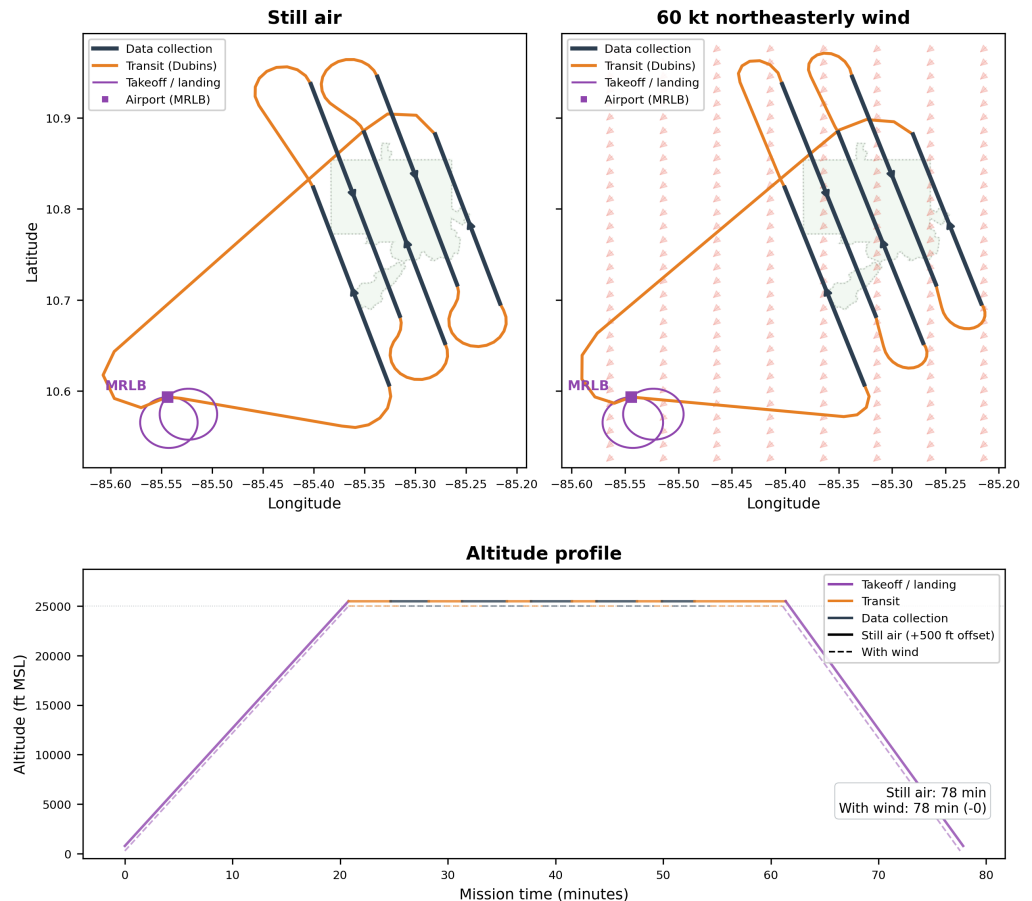


Figure 2: Wind-aware mission planning over Rincón de la Vieja. Top: map views comparing still-air Dubins transit arcs (left) with trochoidal arcs under 60 kt northeasterly wind (right). Bottom: altitude profile showing segment-by-segment timing for both conditions. Wind increases total mission time and distorts transit geometry, effects that must be accounted for in fuel planning and airspace coordination.

Composable abstractions rather than monolithic workflows

HyPlan decomposes the planning problem into **composable abstractions** that can be combined into flexible workflows. Core abstractions include flight lines, survey patterns, sensor models, swath generators, glint analyses, and sortie/mission planners.

Each component is designed to operate independently while maintaining compatibility with others. For example, *FlightLine* objects can be generated independently of sensor choice, sensor models can be applied to arbitrary flight geometries, and swath or glint analyses can be layered onto the same flight lines without modifying the underlying planning objects.

The trade-off is that users must assemble workflows themselves rather than relying on a single “one-click” planner. However, this modular design allows HyPlan to support a wide range of research scenarios, from simple coverage estimation to complex multi-day campaign optimization. It also enables extensibility: new aircraft, sensors, flight patterns, or analysis methods can be introduced without modifying core modules.

131 Explicit units and geodesic accuracy

132 All physical quantities in HyPlan carry explicit units using the Pint library ([Grecco, 2024](#)),
133 and all spatial calculations are performed on the WGS84 ellipsoid using geodesic methods
134 implemented via pymap3d ([Hirsch, 2018](#)) and pyproj ([Snow & others, 2023](#)).

135 This design avoids the class of unit conversion and projection errors that commonly arise in
136 mixed-domain workflows—the kind of error famously responsible for the loss of Mars Climate
137 Orbiter ([NASA, 1999](#)). The trade-off is additional implementation complexity compared to
138 unitless or planar approaches. The benefit is **numerical correctness and consistency across**
139 **domains**, allowing seamless integration of aviation conventions (feet, knots, nautical miles)
140 with scientific conventions (meters, m/s, kilometers).

141 For airborne campaigns spanning hundreds to thousands of kilometers, these geodesic
142 considerations are essential to maintaining spatial accuracy.

143 Integration of flight execution and environmental timing

144 A distinguishing design feature of HyPlan is the integration of **flight execution modeling** (“how
145 to fly”) with **environmental timing constraints** (“when to fly”) within a single framework.

146 In addition to geometric and aircraft modeling, HyPlan includes modules for:

- 147 ▪ **Solar geometry and glint analysis** (sun, glint): computation of solar elevation ([Reda](#)
148 [& Andreas, 2004](#)) and specular reflection geometry, including glint arcs that constrain
149 flight heading and timing
- 150
- 151 ▪ **Cloud climatology** (clouds): estimation of clear-sky probability from reanalysis data via
152 Open-Meteo ([Open-Meteo, 2024](#)) or Google Earth Engine ([Gorelick et al., 2017](#)), with
153 campaign simulation tools
- 154
- 155 ▪ **Vegetation phenology** (phenology): extraction of NDVI/LAI time series and phenological
156 transition dates from MODIS products
- 157
- 158 ▪ **Satellite coordination** (satellites): prediction of overpass timing and swath overlap
159 using TLE propagation via Skyfield ([Rhodes, 2019](#)) and CelesTrak ([CelesTrak, 2024](#))

160 These modules enable users to determine not only where and how to fly, but also **when**
161 **environmental conditions are suitable for data collection**. The trade-off is increased system
162 scope compared to planners focused solely on geometry or navigation. However, airborne
163 campaigns are inherently constrained by environmental conditions, and optimal data acquisition
164 depends on jointly satisfying spatial and temporal constraints.

165 Logistics-aware mission planning

166 HyPlan incorporates logistics considerations directly into the planning process. The
167 `flight_plan` module models complete sorties, while `flight_optimizer` addresses the problem
168 of ordering flight lines across multiple days subject to constraints such as aircraft endurance,
169 maximum daily flight time, and the availability of refueling airports.

170 Airport selection is supported through integration with the OurAirports database ([OurAirports,](#)
171 [2024](#)), enabling filtering by runway length, surface type, and proximity. Airspace conflict
172 detection uses FAA NASR and OpenAIP ([OpenAIP, 2024](#)) data to identify intersections with
173 restricted, prohibited, or controlled airspace. These logistics constraints interact with solar and
174 environmental timing, creating a coupled optimization problem that HyPlan addresses through
175 heuristic graph-based methods using NetworkX ([Hagberg et al., 2008](#)).

176 The trade-off is that the optimizer uses heuristic approaches rather than guaranteeing global
177 optimality. The benefit is **computational efficiency and practical applicability**, producing usable

178 multi-day schedules for large campaigns without requiring expensive optimization algorithms.

179 Design implications for research applications

180 These design choices collectively enable HyPlan to function as a **reproducible research tool**
181 rather than a static planning utility. Researchers can encode mission design assumptions in
182 code, evaluate sensitivity to parameters such as altitude, overlap, or cloud thresholds, and
183 generate planning products that are directly traceable to scientific requirements.

184 This architecture supports a range of applications, including:

- 185 ▪ design of airborne calibration/validation campaigns for satellite missions
- 186 ▪ planning of ecological and biogeochemical surveys with phenology constraints
- 187 ▪ optimization of multi-day campaigns under cloud, solar, and logistics constraints
- 188 ▪ integration of airborne observations with satellite overpasses

192 Limitations

193 HyPlan is designed for pre-campaign planning and does not currently model real-time operational
194 constraints such as dynamic weather avoidance, air traffic control restrictions, or in-flight
195 replanning. These capabilities are typically addressed by operational tools such as Moving
196 Lines during campaign execution.

197 Aircraft performance parameters for most platforms are drawn from published specifications
198 and operator-provided values. The NASA ER-2 has been calibrated against 17 in-situ flight logs
199 in the Inter-agency Working Group 1 (IWG1) format (~64,000 cruise-altitude fixes), yielding
200 distinct climb / cruise / descent true-airspeed schedules, an 8-anchor climb profile that resolves
201 the 19–21 kft step climb, a 6-anchor descent profile, and an empirical 2.51° terminal-approach
202 glideslope. Equivalent calibration for the rest of the fleet is ongoing; infrastructure for IWG1
203 calibration and ADS-B-based fitting is included in the library.

204 Research Impact Statement

205 Early versions of HyPlan have been applied in exploratory and pre-campaign planning workflows
206 for multiple NASA airborne science campaigns, including the SHIFT campaign ([Chadwick et
207 al., 2025](#)), AVIRIS deployments for the NASA Arctic Boreal Vulnerability Experiment (ABoVE)
208 in 2022 and 2023 ([Miller et al., 2025](#)), the BioSCaPE campaign ([Cardoso et al., 2025](#)), and
209 the AVUELO AVIRIS campaign (2025).

210 In these applications, HyPlan supported evaluation of flight line configurations, terrain and
211 solar constraints, and mission feasibility under logistical and environmental limitations, directly
212 informing the design of core capabilities such as terrain-aware swath modeling, flight pattern
213 generation, and mission-level scheduling.

214 The software demonstrates strong community-readiness signals, including over 1,200
215 automated tests with greater than 80% code coverage, continuous integration on every
216 commit, comprehensive API documentation, and more than 20 Jupyter notebooks that serve
217 as both tutorials and integration tests. Core calculations are validated against independent
218 references including Vincenty geodesic ([Vincenty, 1975](#)) test cases, NOAA solar geometry
219 calculations ([Reda & Andreas, 2004](#)), and analytical sensor models.

220 HyPlan is under active development for integration into future NASA airborne campaign
221 planning workflows, where its ability to unify sensor modeling, aircraft performance, and
222 environmental constraints in a single framework is expected to support more reproducible and
223 efficient mission design.

224 AI Usage Disclosure

225 Generative AI tools (Claude and ChatGPT) were used to assist with drafting and editing
226 this manuscript. The software itself was developed with AI coding assistance (Claude and
227 ChatGPT). All AI-generated content was reviewed and verified by the author.

228 Acknowledgements

229 The author thanks Samuel LeBlanc for developing Moving Lines and for contributions to the
230 airborne science planning community that informed HyPlan's design.

231 This work was supported by the National Aeronautics and Space Administration.

232 References

- 233 Cardoso, A. W., Hestir, E. L., Slingsby, J. A., & others. (2025). The biodiversity survey of the
234 Cape (BioSCape), integrating remote sensing with biodiversity science. *Npj Biodiversity*,
235 4(1), 5. <https://doi.org/10.1038/s44185-024-00071-5>
- 236 Celestrak. (2024). *Celestrak: Current GP element sets*. <https://celestrak.org/>
- 237 Chadwick, K. D., Cawse-Nicholson, K., Thompson, D. R., & others. (2025). Unlocking
238 ecological insights from sub-seasonal visible-to-shortwave infrared imaging spectroscopy:
239 The SHIFT campaign. *Ecosphere*, 16(3), e70194. <https://doi.org/10.1002/ecs2.70194>
- 240 Dronecode Project. (2024). *QGroundControl: Intuitive and powerful ground control station
241 for the MAVLink protocol*. <http://qgroundcontrol.com>
- 242 Dubins, L. E. (1957). On curves of minimal length with a constraint on average curvature,
243 and with prescribed initial and terminal positions and tangents. *American Journal of
244 Mathematics*, 79(3), 497–516. <https://doi.org/10.2307/2372560>
- 245 GDAL/OGR contributors. (2024). *GDAL/OGR geospatial data abstraction software library*.
246 Open Source Geospatial Foundation. <https://doi.org/10.5281/zenodo.5884351>
- 247 Gillies, S., & contributors. (2024). *Rasterio: Geospatial raster i/o for python*. <https://github.com/rasterio/rasterio>
- 248 Gillies, S., & others. (2007). *Shapely: Manipulation and analysis of geometric objects*. Zenodo.
249 <https://doi.org/10.5281/zenodo.5597138>
- 250 Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017).
251 Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of
252 Environment*, 202, 18–27. <https://doi.org/10.1016/j.rse.2017.06.031>
- 253 Grecco, H. E. (2024). *Pint: Operate and manipulate physical quantities in Python*. <https://pint.readthedocs.io>
- 254 Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics,
255 and function using NetworkX. *Proceedings of the 7th Python in Science Conference*, 11–15.
256 <https://doi.org/10.25080/TCWV9851>
- 257 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
258 Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk,
259 M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant,
260 T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 261 Hirsch, M. (2018). PyMap3D: 3-D coordinate conversions for terrestrial and geospace
262 environments. *Journal of Open Source Software*, 3(23), 580. <https://doi.org/10.21105/>
- 263

266 [joss.00580](#)

267 Jordahl, K., Van den Bossche, J., Fleischmann, M., & others. (2020). *GeoPandas: Python*
268 *tools for geographic data*. Zenodo. <https://doi.org/10.5281/zenodo.3946761>

269 LeBlanc, S. E. (2018). *Moving lines: NASA airborne research flight planning tool*. Zenodo.
270 <https://doi.org/10.5281/zenodo.1478126>

271 McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the*
272 *9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>

273 Miller, C. E., Green, R. O., Thompson, D. R., Thorpe, A. K., Eastwood, M. L., McCubbin,
274 I. B., Olson-Duvall, W., Bernas, M. A., Sarture, C. M., Rios, L. M., Hernandez, M. A.,
275 Bue, B. D., Lundeen, S. R., Pavlick, R., Chapman, J. W., Brodrick, P. G., Eckert, R. F.,
276 Coleman, R. W., Baskaran, L., & Elder, C. D. (2025). Airborne imaging spectroscopy
277 surveys of Arctic and boreal Alaska and northwestern Canada 2017–2023. *Scientific Data*,
278 12, 692. <https://doi.org/10.1038/s41597-025-04898-w>

279 NASA. (1999). *Mars Climate Orbiter mishap investigation board phase I report*. https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf
280

281 OpenAIP. (2024). *OpenAIP: Open aviation data*. <https://www.openaip.net/>

282 Open-Meteo. (2024). *Open-meteo weather API*. <https://open-meteo.com>

283 OurAirports. (2024). *OurAirports: Open data for airports*. <https://ourairports.com/data/>

284 Reda, I., & Andreas, A. (2004). Solar position algorithm for solar radiation applications. *Solar*
285 *Energy*, 76(5), 577–589. <https://doi.org/10.1016/j.solener.2003.12.003>

286 Rhodes, B. (2019). *Skyfield: High precision research-grade positions for planets and Earth*
287 *satellites generator*. <https://rhodesmill.org/skyfield/>

288 Sachdev, S., Moon, B., Yuan, J., & Scherer, S. (2023). Time-optimal path planning in a
289 constant wind for uncrewed aerial vehicles using Dubins set classification. *arXiv Preprint*
290 *arXiv:2306.11845*. <https://doi.org/10.48550/arXiv.2306.11845>

291 Snow, A., & others. (2023). *Pyproj: Python interface to PROJ*. <https://github.com/pyproj4/pyproj>
292

293 Vana, P., Neto, A. A., Faigl, J., & Macharet, D. G. (2020). Minimal 3D Dubins path with
294 bounded curvature and pitch angle. *2020 IEEE International Conference on Robotics and*
295 *Automation (ICRA)*, 8497–8503. <https://doi.org/10.1109/ICRA40945.2020.9197348>

296 Vincenty, T. (1975). Direct and inverse solutions of geodesics on the ellipsoid with application
297 of nested equations. *Survey Review*, 23(176), 88–93. [https://doi.org/10.1179/sre.1975.23.](https://doi.org/10.1179/sre.1975.23.176.88)
298 [176.88](https://doi.org/10.1179/sre.1975.23.176.88)